l_1-K-SVD

Presented by Dwaipayan Haldar

SR Number: 04-03-01-10-42-25-1-27128
Indian Institute of Science, Bengaluru

Original Paper by Dr. Subhadip Mukherjee,
Rupam Basu, and Dr. Chandra Sekhar Seelamantula

Content

1. Why Dictionary?
2. General Algorithm
3. K-SVD
4. l1-KSVD
5. Image Denoising

Content

- Why Dictionary?
- General Algorithm
- K-SVD
- l1-KSVD
- Image Denoising

Content

$$\begin{bmatrix} \text{Why Dictionary?} \\ \text{General Algorithm} \\ \text{K-SVD} \\ \text{I1-KSVD} \\ \text{Image Denoising} \end{bmatrix} = \underbrace{\begin{bmatrix} | & & | & & | & & | & & | \\ | & & | & & | & & | & & | \\ 5 & \cdot & 8 & \cdot & 9 & \cdot & 32 & \cdot & 46 \\ | & & | & & | & & | & & | \\ | & & | & & | & & | & & | \end{bmatrix}}_{\text{Slide number}} \begin{bmatrix} 1 \\ \vdots \\ 1 \\ \vdots \\ 1 \\ \vdots \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

Why Dictionary?

$$y = Ax \quad \text{s.t.} \quad \|x\|_0 \leq T_0$$

$$y \in \mathbb{R}^{m\times 1}, \qquad A \in \mathbb{R}^{m\times n}, \qquad x \in \mathbb{R}^{n\times 1}$$

Why Dictionary?

$$y_i = Dx_i \quad \text{s.t.} \quad \|x_i\|_0 \leq T_0$$

$$y_i \in \mathbb{R}^{m\times 1}, \qquad A \in \mathbb{R}^{m\times n}, \qquad x_i \in \mathbb{R}^{n\times 1}$$

Why Dictionary?

$$\underbrace{\begin{bmatrix} | & | & & | \\ y_1 & y_2 & \cdots & y_N \\ | & | & & | \end{bmatrix}}_{Y_{m\times N}} = D_{m\times n} \underbrace{\begin{bmatrix} | & | & & | \\ x_1 & x_2 & \cdots & x_N \\ | & | & & | \end{bmatrix}}_{X_{n\times N}}$$

Here, $m \ll n \ll N$.

The 2 steps to success

- *Sparse Coding Step:* Finds the coefficients given the dictionary. Use the algorithms we learned in the class.
- *Dictionary Update Step:* The dictionary is updated assuming known and fixed coefficients.

K-SVD

K-SVD - Basic Formulation

$$\min_{D,X} \|Y - DX\|_F^2 \qquad \text{subject to} \qquad \|x_i\|_0 \leq T_0, \quad \forall i$$

$$\text{where } \|\cdot\|_F = \sqrt{\sum_{ij} A_{ij}^2}$$

K-SVD - Sparse Coding Step

Fix the D, then the penalty term can be written as

$$\|Y - DX\|_F^2 = \sum_{i=1}^{N} \|y_i - Dx_i\|_2^2$$

K-SVD - Sparse Coding Step

Fix the D, then the penalty term can be written as

$$\|Y - DX\|_F^2 = \sum_{i=1}^{N} \|y_i - Dx_i\|_2^2$$

So, the minimization problem becomes

$$\min_{x_i} \|y_i - Dx_i\|_2^2 \qquad \text{subject to} \qquad \|x_i\|_0 \leq T_0, \quad \forall i$$

K-SVD - Sparse Coding Step

Fix the D, then the penalty term can be written as

$$\|Y - DX\|_F^2 = \sum_{i=1}^{N} \|y_i - Dx_i\|_2^2$$

So, the minimization problem becomes

$$\min_{x_i} \|y_i - Dx_i\|_2^2 \qquad \text{subject to} \qquad \|x_i\|_0 \leq T_0, \quad \forall i$$

It can be solved with respect to each i with the help of any suitable algorithm taught in the class like OMP, FOCUSS, BP etc.

Detour - The matrix multiplication trivia

$$DX = \begin{bmatrix} | & | & & | \\ d_1 & d_2 & \cdots & d_n \\ | & | & & | \end{bmatrix}_{m \times n} \begin{bmatrix} - & x_1 & - \\ - & x_2 & - \\ & \vdots & \\ - & x_n & - \end{bmatrix}_{n \times N}$$

Detour - The matrix multiplication trivia

$$DX = \begin{bmatrix} \boxed{\begin{matrix} | \\ d_1 \\ | \end{matrix}} & \begin{matrix} | \\ d_2 \\ | \end{matrix} & \cdots & \begin{matrix} | \\ d_n \\ | \end{matrix} \end{bmatrix}_{m \times n} \begin{bmatrix} \boxed{- \; x_1 \; -} \\ - \; x_2 \; - \\ \vdots \\ - \; x_n \; - \end{bmatrix}_{n \times N}$$

Detour - The matrix multiplication trivia

$$DX = \begin{bmatrix} \boxed{\begin{matrix} | \\ d_1 \\ | \end{matrix}} & \begin{matrix} | \\ d_2 \\ | \end{matrix} & \cdots & \begin{matrix} | \\ d_n \\ | \end{matrix} \end{bmatrix}_{m \times n} \begin{bmatrix} \boxed{- \; x_1 \; -} \\ - \; x_2 \; - \\ \vdots \\ - \; x_n \; - \end{bmatrix}_{n \times N}$$

$$\equiv \begin{bmatrix} d_1 x_1 \end{bmatrix}_{m \times N}$$

Detour - The matrix multiplication trivia

$$DX = \begin{bmatrix} | & \boxed{\begin{matrix} | \\ d_2 \\ | \end{matrix}} & \cdots & | \\ d_1 & & & d_n \\ | & & & | \end{bmatrix}_{m \times n} \begin{bmatrix} - & x_1 & - \\ \boxed{- \; x_2 \; -} \\ & \vdots & \\ - & x_n & - \end{bmatrix}_{n \times N}$$

$$\equiv \begin{bmatrix} d_1 x_1 \end{bmatrix}_{m \times N} + \begin{bmatrix} d_2 x_2 \end{bmatrix}_{m \times N}$$

Detour - The matrix multiplication trivia

$$DX = \begin{bmatrix} | & | & & \boxed{|} \\ d_1 & d_2 & \cdots & \boxed{d_n} \\ | & | & & \boxed{|} \end{bmatrix}_{m\times n} \begin{bmatrix} - & x_1 & - \\ - & x_2 & - \\ & \vdots & \\ \boxed{-} & \boxed{x_n} & \boxed{-} \end{bmatrix}_{n\times N}$$

$$\equiv \begin{bmatrix} d_1 x_1 \end{bmatrix}_{m\times N} + \begin{bmatrix} d_2 x_2 \end{bmatrix}_{m\times N} + \cdots + \begin{bmatrix} d_n x_n \end{bmatrix}_{m\times N}$$

Back to K-SVD

Now, we know the X matrix obtained by doing the sparse coding step. We need to update the D. That we will do column by column. So, the penalty term can be written as

$$\|Y - DX\|_F^2 = \left\| \left(Y - \sum_{j \neq k} d_j x_j \right) - d_k x_k \right\|_F^2$$

K-SVD - Dictionary Update Step

Now, we know the X matrix obtained by doing the sparse coding step. We need to update the D. That we will do column by column. So, the penalty term can be written as

$$\|Y - DX\|_F^2 = \left\| \left(Y - \sum_{j \neq k} d_j x_j \right) - d_k x_k \right\|_F^2 = \|E_k - d_k x_k\|_F^2$$

K-SVD - Dictionary Update Step

Now, we know the X matrix obtained by doing the sparse coding step. We need to update the D. That we will do column by column. So, the penalty term can be written as

$$\|Y - DX\|_F^2 = \left\| \left(Y - \sum_{j \neq k} d_j x_j \right) - d_k x_k \right\|_F^2 = \|E_k - d_k x_k\|_F^2$$

Here, E_k stands for the error for all the N examples when the kth atom is removed.

K-SVD - Dictionary Update Step

So, here is the easy solution:

- Do SVD of E_k, $E_k = U \Delta V^T$.

K-SVD - Dictionary Update Step

So, here is the easy solution:

- Do SVD of E_k, $E_k = U \Delta V^T$.
- Select d_k as the first column of U and x_k as the first column of V multiplied by $\Delta(1,1)$.
- That effectively would give the closest rank 1 approximation via Eckart–Young–Mirsky Theorem (Appendix).

K-SVD - Dictionary Update Step

So, here is the easy solution:

- Do SVD of E_k, $E_k = U \Delta V^T$.
- Select d_k as the first column of U and x_k as the first column of V multiplied by $\Delta(1,1)$.
- That effectively would give the closest rank 1 approximation via Eckart–Young–Mirsky Theorem (Appendix).

Are we done?

K-SVD - Dictionary Learning Step

Yes, we missed the sparsity. Here is the real solution.

- Define $\omega_k = \{i \mid 1 \leq i \leq n,\ x_k(i) \neq 0\}$

K-SVD - Dictionary Learning Step

Yes, we missed the sparsity. Here is the real solution.

- Define $\omega_k = \{i \mid 1 \leq i \leq n,\ x_k(i) \neq 0\}$
- Define Ω_k as a matrix of size $N \times |\omega_k|$ with ones on the $(\omega_k(i), i)$-th entries and zeros elsewhere.

K-SVD - Dictionary Learning Step

Yes, we missed the sparsity. Here is the real solution.

- Define $\omega_k = \{i \mid 1 \leq i \leq n,\ x_k(i) \neq 0\}$
- Define Ω_k as a matrix of size $N \times |\omega_k|$ with ones on the $(\omega_k(i), i)$-th entries and zeros elsewhere.
- Change the minimization problem

$$\|E_k - d_k x_k\|_F^2 \rightarrow \|E_k \Omega_k - d_k x_k \Omega_k\|_F^2 = \left\| E_k^R - d_k x_k^R \right\|_F^2$$

- Apply the same SVD thing over this modified optimization problem.

K-SVD - Dictionary Update Step

So, here is the final solution:

- Do SVD of E_k^R, $E_k^R = U \Delta V^T$.
- Select d_k as the first column of U and x_k^R as the first column of V multiplied by $\Delta(1,1)$.
- That effectively would give the closest rank 1 approximation via Eckart–Young–Mirsky Theorem (Appendix).

Yes, we are done

K-SVD - Remarks

- The columns of D remain normalized after each update.

K-SVD - Remarks

- The columns of D remain normalized after each update.
- The support of all the columns of X stays either same or gets smaller after each update.

K-SVD - Remarks

- The columns of D remain normalized after each update.
- The support of all the columns of X stays either same or gets smaller after each update.
- There is simultaneous update of both the X and D in the dictionary update step. Consider two cases, one without simultaneous update and other with simultaneous update.

K-SVD - Remarks

- The columns of D remain normalized after each update.
- The support of all the columns of X stays either same or gets smaller after each update.
- There is simultaneous update of both the X and D in the dictionary update step. Consider two cases, one without simultaneous update and other with simultaneous update.

$$\hat{X}^{\text{wsu}} = \hat{X}, \hat{D}^{\text{wsu}} = \arg\min_{D} J(D, \hat{X}) \quad \text{s.t. } \|d_j\|_2 = 1 \text{ for all } j \tag{1}$$

$$\hat{D}^{\text{su}}, \hat{X}^{\text{su}} = \arg\min_{D,X} J(D, X) \quad \text{s.t. } \|d_j\|_2 = 1 \text{ for all } j, \ \text{supp}(X) \subseteq \Omega \tag{2}$$

$$J(\hat{D}^{\text{su}}, \hat{X}^{\text{su}}) \leq J(\hat{D}^{\text{wsu}}, \hat{X}^{\text{wsu}})$$

Case 1 has lesser search space and Case 2. So case 2 is likely to end up in a lesser minima than case 1.

l_1K-SVD

l_1-K-SVD - Basic Formulation

$$\min_{D,x_i} \sum_{i=1}^{N} \|y_i - Dx_i\|_1 + \sum_{i=1}^{N} \lambda_i \|x_i\|_1 \qquad \text{s.t. } \|d_j\|_2 = 1 \text{ for all } j$$

l_1-K-SVD - Sparse Coding Step

$$\min_{x_i} \left\| y_i - \hat{D} x_i \right\|_1 + \lambda_i \|x_i\|_1$$

l_1-K-SVD - Sparse Coding Step

$$\min_{x_i} \left\| y_i - \hat{D} x_i \right\|_1 + \lambda_i \|x_i\|_1$$

The update of x_i in the kth iteration is given as:

$$x_i^{(k+1)} = \left(\hat{D}^T W_{1i}^{(k)} \hat{D} + \lambda_i W_{2i}^{(k)} \right)^{-1} \hat{D}^T W_{1i}^{(k)} y_i$$

l_1-K-SVD - Sparse Coding Step

$$\min_{x_i} \left\| y_i - \hat{D} x_i \right\|_1 + \lambda_i \|x_i\|_1$$

The update of x_i in the kth iteration is given as:

$$x_i^{(k+1)} = \left(\hat{D}^T W_{1i}^{(k)} \hat{D} + \lambda_i W_{2i}^{(k)} \right)^{-1} \hat{D}^T W_{1i}^{(k)} y_i$$

W_{1i} and W_{2i} are diagonal weight matrices that are given by

$$W_{1i}^{(k+1)}(j) = \frac{1}{\left| (y_i - \hat{D} x_i^{(k)})_j \right| + \epsilon}, \qquad W_{2i}^{(k+1)}(j) = \frac{1}{\left| (x_i^{(k)})_j \right| + \epsilon}$$

l_1-K-SVD - Dictionary Update Step

$$\min_{D,X} \|Y - DX\|_1 \qquad \text{s.t. } \|d_j\|_2 = 1 \text{ for all } j,\ \text{supp}(X) \subseteq \Omega$$

$$\text{where } \|\cdot\|_1 = \sum_{i=1}^{N} \|y_i - Dx_i\|_1$$

l_1-K-SVD - Dictionary Update Step

$$\min_{d_k, x_k} \|E_k - d_k x_k\|_1 \qquad \text{s.t. } \operatorname{supp}(x_k) = \omega_k, \ \|d_k\|_2 = 1$$

l_1-K-SVD - Dictionary Update Step

$$\min_{d_k, x_k} \|E_k - d_k x_k\|_1 \qquad \text{s.t. } \operatorname{supp}(x_k) = \omega_k, \ \|d_k\|_2 = 1$$

The support can be included in the optimization problem similar to K-SVD with something like this

$$\|E_k \Omega_k - d_k x_k \Omega_k\|_1 = \|E_k^R - d_k x_k^R\|_1$$

l_1-K-SVD - Dictionary Update Step

$$\min_{d_k, x_k} \|E_k - d_k x_k\|_1 \qquad \text{s.t. } \operatorname{supp}(x_k) = \omega_k, \ \|d_k\|_2 = 1$$

The support can be included in the optimization problem similar to K-SVD with something like this

$$\|E_k \Omega_k - d_k x_k \Omega_k\|_1 = \|E_k^R - d_k x_k^R\|_1$$

Further simplifying the notation we can say the problem statement as where we need to find the value of u and v.

$$\min_{u,v} \|E - uv^T\|_1 = \min_{u,v} \sum_{i=1}^{N} \|e_i - v_i u\|_1 \qquad \text{s.t. } \|u\|_2 = 1$$

l_1-K-SVD - Dictionary Update Step

That l_1 norm can be approximated by reweighted l_2 metric

$$\sum_{i=1}^{N} \|e_i - v_i u\|_1 \approx \sum_{i=1}^{N} (e_i - v_i u)^T W_i (e_i - v_i u)$$

l_1-K-SVD - Dictionary Update Step

That l_1 norm can be approximated by reweighted l_2 metric

$$\sum_{i=1}^{N} \|e_i - v_i u\|_1 \approx \sum_{i=1}^{N} (e_i - v_i u)^T W_i (e_i - v_i u)$$

$$\text{where} W_i(j) = \frac{1}{|(e_i - v_i u)_j| + \epsilon}$$

Differentiating with respect to u and setting to zero for a fixed v

l_1-K-SVD - Dictionary Update Step

That l_1 norm can be approximated by reweighted l_2 metric

$$\sum_{i=1}^{N} \|e_i - v_i u\|_1 \approx \sum_{i=1}^{N} (e_i - v_i u)^T W_i (e_i - v_i u)$$

$$\text{where} W_i(j) = \frac{1}{|(e_i - v_i u)_j| + \epsilon}$$

Differentiating with respect to u and setting to zero for a fixed v

$$u = \left(\sum_{i=1}^{N} v_i^2 W_i \right)^{-1} \left(\sum_{i=1}^{N} v_i W_i e_i \right)$$

Similarily for a fixed u,

$$v_i = \frac{u^T W_i e_i}{u^T W_i u},$$

l_1-K-SVD - Final Algorithm to find the best u and v

- Input: Error matrix E, number of iterations J.
- Initialization: Set the iteration counter $t = 0$, $u^{(t)} = a$, and $v^{(t)} = \sigma b$, where a and b are the left and right singular vectors corresponding to the largest singular value σ of E.
- Initialize the weight matrices $W_i^{(t)} = I$, for $i = 1, 2, \ldots, N$.
- Iterate the following steps J times: Step 1:

$$W_i^{(t+1)}(j) = \frac{1}{|(e_i - v_i^{(t)} u^{(t)})_j| + \epsilon}$$

Step 2:

$$u^{(t+1)} = \Big(\sum_{i=1}^{N} (v_i^{(t)})^2 W_i^{(t+1)} \Big)^{-1} \Big(\sum_{i=1}^{N} v_i^{(t)} W_i^{(t+1)} e_i \Big)$$

Step 3:

$$v_i^{(t+1)} = \frac{(u^{(t+1)})^T W_i e_i}{(u^{(t+1)})^T W_i (u^{(t+1)})}$$

- Scaling:

$$d_k = \frac{u^{(t)}}{\|u^{(t)}\|_2}, \qquad x_k^R = \|u^{(t)}\|_2 (v^{(t)})^T$$

l_1-K-SVD - Remarks

- **Pruning step (missing from the main algorithm):** After each sparse coding step, any coefficient satisfying

$$|x_i(j)| < T_0 \cdot \|X\|_F$$

is pruned to zero. The paper uses $T_0 = 0.03$.

l_1-K-SVD - Remarks

- **Pruning step (missing from the main algorithm):** After each sparse coding step, any coefficient satisfying

$$|x_i(j)| < T_0 \cdot \|X\|_F$$

  is pruned to zero. The paper uses $T_0 = 0.03$.
- **Computationally expensive step:** The dictionary update (Algorithm 1) is the bottleneck: K atoms $\times$ J inner IRLS iterations, each an $m \times m$ linear solve. $\Rightarrow$ $\mathcal{O}(K \cdot J \cdot m^3)$ work per outer iteration — significantly heavier than the single SVD per atom in K-SVD.

Image Denoising — The Core Idea

- **Corrupted observation:** $y = Dx + w$, where w is noise (Gaussian, Laplacian, or missing pixels).

Image Denoising — The Core Idea

- **Corrupted observation:** $y = Dx + w$, where w is noise (Gaussian, Laplacian, or missing pixels).
- **Learn the dictionary D:** Train on patches so that D captures dominant patterns — edges, textures, colour gradients.

Image Denoising — The Core Idea

- **Corrupted observation:** $y = Dx + w$, where w is noise (Gaussian, Laplacian, or missing pixels).
- **Learn the dictionary D:** Train on patches so that D captures dominant patterns — edges, textures, colour gradients.
- **Sparse coding:** Given D and noisy patch y, solve

$$\hat{x} = \arg\min_{x} \ \|y - Dx\|_1 + \lambda \|x\|_1$$

ℓ_1 fidelity down-weights outlier pixels.

Image Denoising — The Core Idea

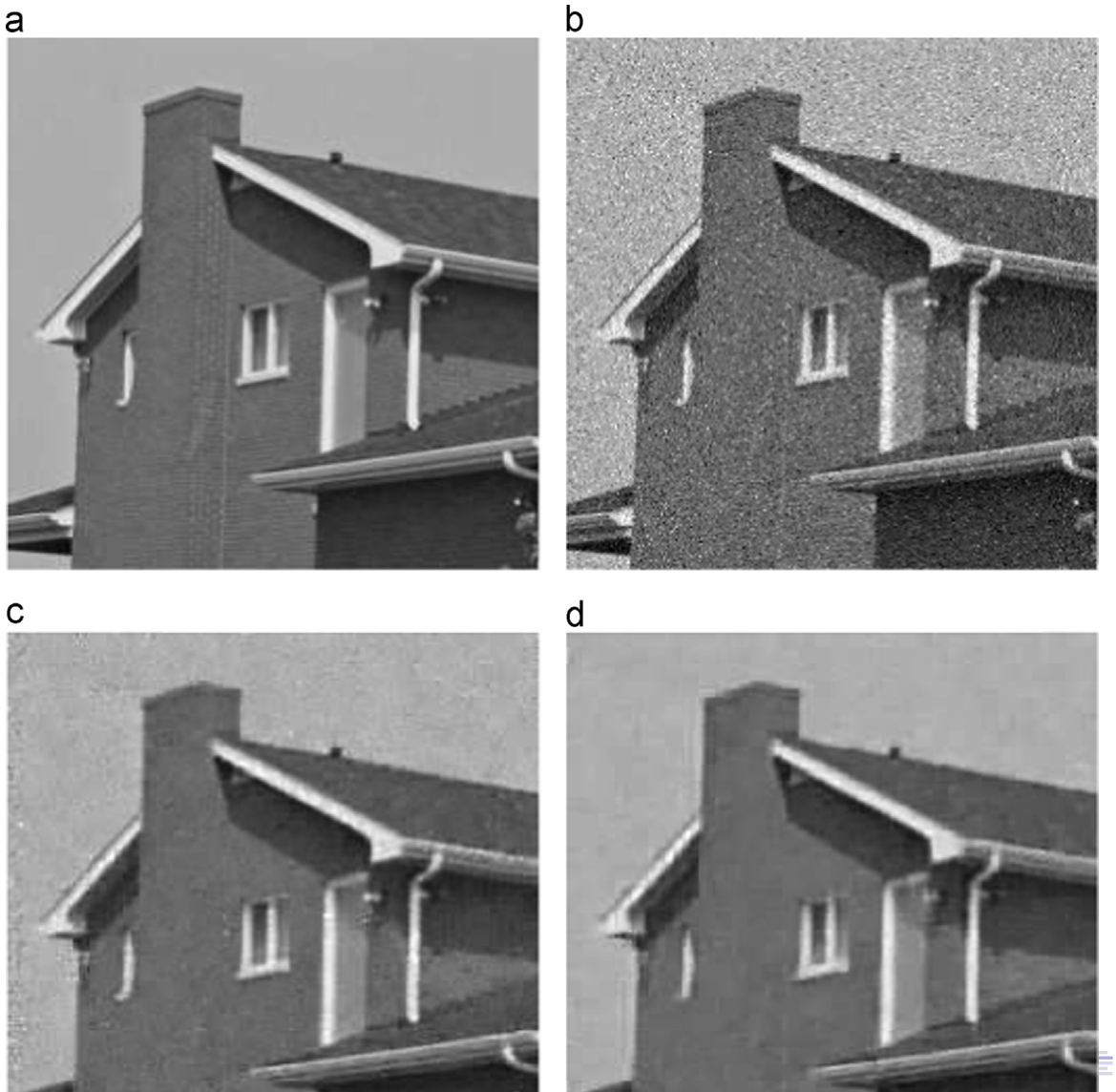
- **Corrupted observation:** $y = Dx + w$, where w is noise (Gaussian, Laplacian, or missing pixels).
- **Learn the dictionary** D**:** Train on patches so that D captures dominant patterns — edges, textures, colour gradients.
- **Sparse coding:** Given D and noisy patch y, solve

$$\hat{x} = \arg\min_{x} \ \|y - Dx\|_1 + \lambda \|x\|_1$$

ℓ_1 fidelity down-weights outlier pixels.

Reconstruct: $\hat{y} = D\hat{x}$ — sparse $\hat{x}$ discards the noise component.

Paper Result — Figure 6 (Mukherjee et al. 2016)



Simulation Result A — Gaussian Noise ($\sigma = 20$)

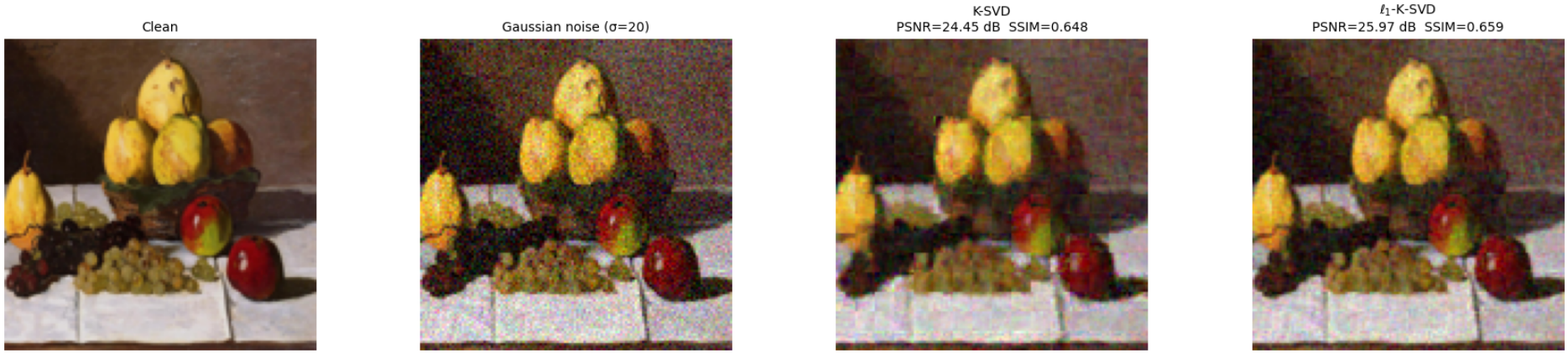


Figure 1: 4-panel comparison on an impressionist painting. Left to right: clean image, noisy input ($\sigma = 20/255$), K-SVD reconstruction, ℓ_1-K-SVD reconstruction. Both methods perform similarly under Gaussian noise, consistent with the ℓ_2 optimality of K-SVD in this regime.

Simulation Result B — Laplacian Noise (same RMS power)

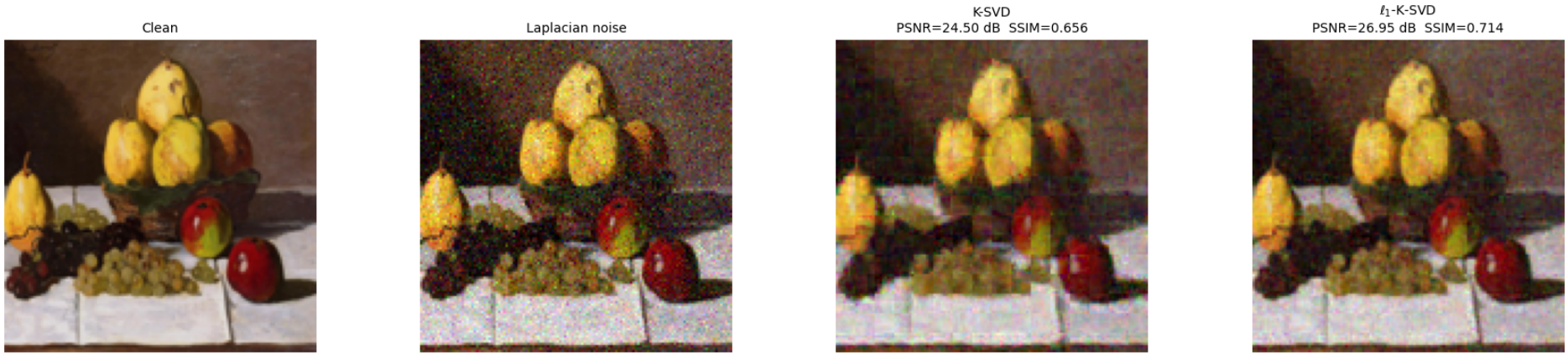


Figure 2: Same test image corrupted with Laplacian noise at the same RMS power as the Gaussian case. ℓ_1-K-SVD achieves higher PSNR and SSIM because its ℓ_1 data-fidelity term is the natural MAP estimator under a Laplacian noise model, while K-SVD's ℓ_2 criterion is sensitive to heavy tails.

Simulation Result C — Inpainting (20% Pixels Removed)

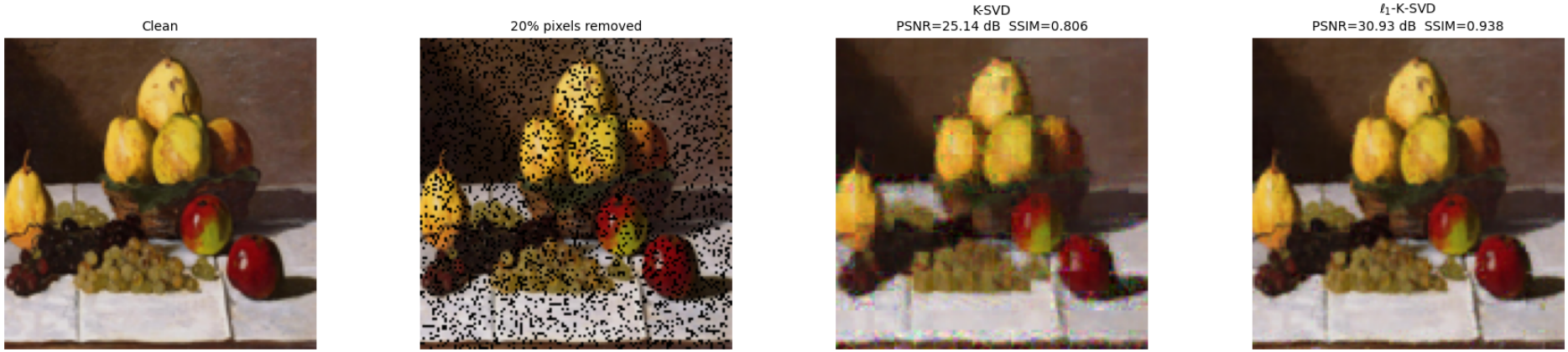


Figure 3: 20% of pixels randomly zeroed before reconstruction. Missing-data-aware sparse coding (fitting only over observed entries) allows ℓ_1-K-SVD to recover fine texture and colour with noticeably higher SSIM than K-SVD.

Appendix

Appendix A: Proof of ℓ_1 Sparse Coding Step

Problem:

$$\min_x \ \|y - Dx\|_1 + \lambda \|x\|_1$$

Let $r_j = (y - Dx)_j$. Objective $= \sum_j |r_j| + \lambda \sum_i |x_i|$

Appendix A: Proof of ℓ_1 Sparse Coding Step

Problem:

$$\min_x \ \|y - Dx\|_1 + \lambda \|x\|_1$$

IRLS approximation:

$$|z| \approx \frac{z^2}{|z^{(k)}| + \epsilon}$$

Define diagonal weight matrices:

$$W_1^{(k)}(j) = \frac{1}{|r_j^{(k)}| + \epsilon}, \qquad W_2^{(k)}(i) = \frac{1}{|x_i^{(k)}| + \epsilon}$$

Appendix A: Proof of ℓ_1 Sparse Coding Step

IRLS weights:

$$W_1^{(k)}(j) = \frac{1}{|r_j^{(k)}| + \epsilon}, \qquad W_2^{(k)}(i) = \frac{1}{|x_i^{(k)}| + \epsilon}$$

So $\sum_j |r_j| \approx r^T W_1 r$ and $\sum_i |x_i| \approx x^T W_2 x$. **Surrogate quadratic:**

$$\min_x \, (y - Dx)^T W_1 (y - Dx) + \lambda \, x^T W_2 x$$

Expand:

$$= \; x^T \left(D^T W_1 D + \lambda W_2 \right) x - 2x^T D^T W_1 y + \text{const}$$

Appendix A: Proof of ℓ_1 Sparse Coding Step

Normal equations (set gradient to zero):

$$\left(D^T W_1^{(k)} D + \lambda W_2^{(k)}\right) x = D^T W_1^{(k)} y$$

Closed-form update:

$$\boxed{x^{(k+1)} = \left(D^T W_1^{(k)} D + \lambda W_2^{(k)}\right)^{-1} D^T W_1^{(k)} y}$$

Weight updates (from definitions):

$$W_1^{(k+1)}(j) = \frac{1}{\left|(y - Dx^{(k)})_j\right| + \epsilon}, \qquad W_2^{(k+1)}(i) = \frac{1}{\left|x_i^{(k)}\right| + \epsilon}$$

Appendix B: Motivation for the ℓ_1 Objective

Bayesian setup. Assume observations follow $y_n = Dx_n + w_n$.

Laplacian noise model:

$$p(\mathbf{w}_n) = \prod_{j=1}^{m} \frac{1}{2\gamma_n} \exp\left(-\frac{|(w_n)_j|}{\gamma_n}\right)$$

Appendix B: Motivation for the ℓ_1 Objective

Laplacian noise:

$$p(\mathbf{w}_n) = \prod_{j=1}^{m} \frac{1}{2\gamma_n} \exp\left(-\frac{|(w_n)_j|}{\gamma_n}\right)$$

Laplacian prior on coefficients:

$$p(\mathbf{x}_n) = \prod_{j=1}^{K} \frac{1}{2\rho_n} \exp\left(-\frac{|(x_n)_j|}{\rho_n}\right)$$

Appendix B: Motivation for the ℓ_1 Objective

Laplacian noise: $p(\mathbf{w}_n) \propto \exp(-\|\mathbf{w}_n\|_1/\gamma_n)$

Laplacian prior: $p(\mathbf{x}_n) \propto \exp(-\|\mathbf{x}_n\|_1/\rho_n)$

MAP estimation: maximise $\log p(\mathbf{x}_n|\mathbf{y}_n)$

$$\Rightarrow \ \arg\min_{\mathbf{x}_n} \ \frac{1}{\gamma_n}\|\mathbf{y}_n - D\mathbf{x}_n\|_1 + \frac{1}{\rho_n}\|\mathbf{x}_n\|_1$$

Setting $\lambda_n = \gamma_n/\rho_n$ and summing over all n:

$$\boxed{\min_{D,\,\mathbf{x}_n} \ \sum_{n=1}^{N} \|\mathbf{y}_n - D\mathbf{x}_n\|_1 + \lambda_n\|\mathbf{x}_n\|_1}$$

This is exactly the ℓ_1-K-SVD objective (Eq. 1 in the paper).

Appendix C: The ℓ_p-K-SVD Algorithm

Generalised objective $(0 < p \leq 1)$:

$$\min_{D,\,\mathbf{x}_n} \sum_{n=1}^{N} \|\mathbf{y}_n - D\mathbf{x}_n\|_p^p + \lambda_n \|\mathbf{x}_n\|_p^p \quad \text{s.t. } \|d_j\|_2 = 1$$

Appendix C: The ℓ_p-K-SVD Algorithm

Generalised objective:

$$\min_{D,\,\mathbf{x}_n} \sum_{n=1}^{N} \|\mathbf{y}_n - D\mathbf{x}_n\|_p^p + \lambda_n \|\mathbf{x}_n\|_p^p$$

Generalized p-Gaussian Bayesian interpretation:

$$p_{\mathrm{gen}}(\mathbf{u}) \propto \prod_j \exp\left(-\frac{|u_j|^p}{p\,\sigma_n^p}\right)$$

MAP maximisation $\Rightarrow$ the ℓ_p^p objective above.

Appendix C: The ℓ_p-K-SVD Algorithm

Modified IRLS weights (sparse coding, replace $|\cdot|$ with $|\cdot|^{2-p}$):

$$W_{1n}^{(k+1)}(j) = \frac{1}{\left|(\mathbf{y}_n - \hat{D}\mathbf{x}_n^{(k)})_j\right|^{2-p} + \epsilon}, \quad W_{2n}^{(k+1)}(j) = \frac{1}{\left|(x_n^{(k)})_j\right|^{2-p} + \epsilon}$$

Modified dictionary update weight (Algorithm 1, Step 1):

$$W_n^{(t+1)}(j) = \frac{1}{\left|(\mathbf{e}_n - v_n\mathbf{u}^{(t)})_j\right|^{2-p} + \epsilon}$$

At $p = 1$: recovers ℓ_1-K-SVD exactly.
At $0 < p < 1$: even sparser solutions, better for impulsive noise.

Appendix D: K-SVD as a Generalisation of K-Means

Vector Quantisation (K-means) problem:

$$\min_{C,X} \|Y - CX\|_F^2 \quad \text{s.t. } x_i = e_k \text{ for some } k$$

e_k is the k-th standard basis vector (one-hot). Each signal is represented by *exactly one* codeword column of C.

Appendix D: K-SVD as a Generalisation of K-Means

K-means:

$$\min_{C,X} \|Y - CX\|_F^2 \quad \text{s.t. } x_i = e_k \;\Rightarrow\; \|x_i\|_0 = 1$$

Limitation: only one prototype contributes per signal.

K-SVD generalisation: relax the one-hot constraint

$$\min_{D,X} \|Y - DX\|_F^2 \quad \text{s.t. } \|x_i\|_0 \leq T_0, \quad \forall\, i$$

Allow up to T_0 dictionary atoms to contribute — sparse, not binary.

Appendix D: K-SVD as a Generalisation of K-Means

K-means: $\|x_i\|_0 = 1$, coefficients are 1, codeword columns are means.

K-SVD: $\|x_i\|_0 \leq T_0$, coefficients are real-valued, atoms updated via SVD.

Same alternating structure:

- *Sparse coding step*: assign signals to atoms (K-means: nearest neighbour; K-SVD: OMP/BP)
- *Dictionary update step*: update each atom/codeword given current assignments

K-SVD is to K-means as $\ell_0 \leq T_0$ is to $\ell_0 = 1$.

Appendix E: Eckart–Young–Mirsky Theorem

Theorem (Eckart–Young 1936, Mirsky 1960).

Let $A \in \mathbb{R}^{m \times n}$ with SVD

$$A = U \Sigma V^T, \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0, \quad p = \min(m, n).$$

The rank-r matrix that minimises $\|A - B\|_F$ is

$$A_r = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^T$$

and the minimum error is $\|A - A_r\|_F = \sqrt{\sigma_{r+1}^2 + \cdots + \sigma_p^2}$.

Appendix E: Eckart–Young–Mirsky Theorem

Theorem. $A_r = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ minimises $\|A - B\|_F$ over $\mathrm{rank}(B) \leq r$.

Step 1 — Unitary invariance. For any unitary U, V:
$\|A - B\|_F = \|U^T(A - B)V\|_F$.
Writing $A = U\Sigma V^T$:

$$\|A - B\|_F = \|\Sigma - C\|_F, \qquad C = U^T B V, \quad \mathrm{rank}(C) = \mathrm{rank}(B) \leq r.$$

Appendix E: Eckart–Young–Mirsky Theorem

Step 1. $\|A - B\|_F = \|\Sigma - C\|_F$ where $C = U^T BV$, $\mathrm{rank}(C) \leq r$.

Step 2 — Optimise over rank-r matrices C.

$$\|\Sigma - C\|_F^2 = \sum_i (\sigma_i - C_{ii})^2 + \sum_{i \neq j} C_{ij}^2$$

Off-diagonal terms $C_{ij}^2 \geq 0$ only increase the objective.
$\Rightarrow$ optimal C is *diagonal*: $C = \mathrm{diag}(c_1, \ldots, c_p)$.

Appendix E: Eckart–Young–Mirsky Theorem

Step 2. Optimal C is diagonal: $\|\Sigma - C\|_F^2 = \sum_i (\sigma_i - c_i)^2$, $\mathrm{rank}(C) \leq r$.

Step 3 — Solve the diagonal problem.
At most r of the c_i can be nonzero. For each active c_i: set $c_i = \sigma_i$ (contributes 0). For inactive $c_i = 0$: contributes σ_i^2.

To minimise total: activate the r *largest* singular values.

$$c_i = \sigma_i \text{ for } i \leq r, \quad c_i = 0 \text{ for } i > r.$$

Conclusion:

$$C^* = \mathrm{diag}(\sigma_1, \ldots, \sigma_r, 0, \ldots, 0) \;\Rightarrow\; B^* = UC^*V^T = A_r$$

$$\|A - A_r\|_F = \sqrt{\sigma_{r+1}^2 + \cdots + \sigma_p^2} \qquad \square$$